



科研训练三

2 论文阅读与调研

晏潇 博士

yanxiaosunny@whu.edu.cn



读论文的困惑

- ❖ 「论文都看了，研究还是不知道怎么做」
 - ❑ 目的不清：读了大量论文，但对研究的推进作用不明显
 - ❑ 没有思考：困在论文的逻辑里，没有形成自己的判断，找不出文章的问题
 - ❑ 缺少比较：关注单篇论文，没有多篇论文横向对比和总结
 - ❑ 只读不动手：不动手进行实现和实验，缺乏第一手经验
- ❖ 本讲目标：把「读过」推进到「能理解·能质疑·能比较·能发现问题」
 - ❑ 阅读，思考，动手三位一体



调研流程：找 → 筛 → 读 → 懂 → 比 → 问

❖ 六步循环，每一步都形成文字，用写来驱动想清楚

□ 找 → 阅读目标 | 筛 → 搜索记录 | 读 → 阅读笔记

□ 懂 → 实现或讲解 | 比 → 比较表 | 问 → 问题描述



阅读目标



阅读目标：文献阅读和教材学习有什么不同？

❖ 教材学习 · 知识覆盖

- 目标：把一块知识系统地过一遍 | 路径：按章节往下推
- 深度：各章差不多，平均分配 | 评价：考过就算完

❖ 文献阅读 · 任务驱动

- 目标：解决手头这个具体问题 | 路径：哪篇相关就看哪篇
- 深度：跟问题越相关读得越深 | 评价：问题、证据、贡献经不经得起推敲

❖ 三个差别

- 提取有用的内容，不用第一页读到最后一页
- 差异化阅读，有的论文 5 分钟就够，有的要花 5 小时
- 读完要有思考、判断或问题



按需学习：遇到障碍再定向补背景

❖ 五步循环：研究任务 → 开始阅读 → 发现障碍 → 定向补课 → 返回任务

- 先抓主干，卡在术语、理论或工具上再停下来补

- 只补必要部分，补完立刻回到原来的阅读产出

❖ 什么时候算补课完成

- 理解这个问题方向上核心论文的论述

❖ 警惕「无限补课」陷阱

- 越补越多，记不清最初要回答什么；把补课当成任务，从未回到主问题

- 补着补着超出任务范围，时间就这么没了

❖ 明确阅读目标

- 目的：这次是为选题、复现、做方案，还是写东西？

- 核心问题：读完必须能回答哪些具体问题

论文检索



论文检索：把调研内容细化为可检索的关键词

❖ 五类关键词：对象 + 任务 + 方法 + 场景 + 指标

- ❑ 对象：cluster computing · distributed system · big data
- ❑ 任务：iterative analytics · interactive query · graph processing
- ❑ 方法：in-memory caching · dataflow · lineage · BSP
- ❑ 场景：large scale · fault tolerant · multi-tenant
- ❑ 指标：latency · I/O overhead · iteration speedup

❖ 组合检索式与单关键词陷阱

- ❑ 示例："iterative data processing" AND "in-memory" AND "fault tolerance"
- ❑ 只搜“big data”，上万篇根本不收敛；只搜 "MapReduce"，大多是应用类，改进类反而少
- ❑ 同义词扩展：in-memory → cache-resident · RAM-based；fault tolerance → resilience · recovery



检索是逐轮收敛，而不是一次命中

❖ 四轮收敛：宽检索 → 引用追踪 → 限定范围 → 稳定集合

- ❑ 第一轮宽检索：摸清这个方向的核心术语，找出综述和经典工作
- ❑ 第二轮引用追踪：向前找改进 / 反例 / 扩展，向后找概念源头
- ❑ 第三轮限定范围：把任务、数据集、指标收紧，加上约束词
- ❑ 第四轮稳定集合：新论文不再带来新维度，涵盖经典 + 新代表作，开始精读



还缺什么？关键词要改什么？ —— 每轮结束前必答的两个问题



种子论文 + 双向引用追踪

❖ 种子论文的三特征

- ❑ 与目标问题高度相关：你写出的核心问题，它就在解决
- ❑ 引用网络丰富：后续工作几乎都引用它，多篇论文拿它当比较基线
- ❑ 种子论文不用是「最好的」，但得是最相关、最被引、最清楚的那一篇

❖ 双向追踪：领域地图沿引用关系逐步长出来

- ❑ 向后追 References —— 找概念来源 / 基础方法 / 被继承的假设
- ❑ 向前追 Cited by —— 找后续改进 / 反例 / 扩展场景 / 新问题
- ❑ 工具：Google Scholar · Semantic Scholar · Connected Papers · DBLP





四类检索来源 + 关键学者

❖ 四类来源各有侧重

- ❑ Google Scholar : 跨领域入口与被引追踪 | DBLP : 会议期刊书目与作者
- ❑ ACM DL / IEEE Xplore : 正式出版版本与元数据 | arXiv : 最新预印本
- ❑ Google Scholar 的关注和提醒功能值得用起来

❖ 关键学者

- ❑ 搞清楚你这个方向、这个问题上有哪些主要的人
- ❑ 定期看看他们最近发了什么、讲了什么

❖ 朋友圈公众号博客

- ❑ 作为跟踪研究方向的补充，不是主要论文来源

论文阅读



阅读价值 = 质量 × 当前相关性

❖ 两个维度决定一篇论文值得投入多少时间

- ❑ 质量：方法是否有代表性、有没有同行评审、论证严不严谨、被引得多不多
- ❑ 相关性：跟目前手上的问题多相关

❖ 四个象限，四种处理方式





三步文献阅读法

❖ 第一遍 · 鸟瞰：5-20 分钟

□看摘要 · 引言 · 章节标题 · 结论，跳过正文细节与证明；判断论文质量，筛掉 90% 论文

❖ 第二遍 · 理解：小时级

□抓住主张、方法和关键证据

❖ 第三遍 · 验证：不限

□复述论文



投入随「论文对当前任务的重要性」递增，而不是随论文字数递增



第一遍 · 鸟瞰：要不要继续读

❖ 四步鸟瞰，5-10 分钟完成

- ❑ 标题 · 摘要 · 引言 —— 这一篇做了什么？
- ❑ 章节与小节标题 —— 结构是实验 / 分析 / 原型 / 理论哪一种？
- ❑ 结论 —— 作者自己怎么说做到了？
- ❑ 参考文献并标记熟悉项 —— 有哪些与我读过的重叠？

❖ 完成标准：能回答「五个 C」

- ❑ Category 论文类型：这篇是测量、分析、原型、理论，还是综述？
- ❑ Context 研究语境：它属于哪个领域，跟哪些已有工作接得上？
- ❑ Correctness 初步正确性：关键假设看起来成立吗？有无反例或明显漏洞？
- ❑ Contributions 主要贡献：作者声称做了什么新的事？是方法、数据还是洞见？
- ❑ Clarity 表达清晰度：论证是否容易跟？图表是否清楚？术语是否定义？



第二遍 · 理解：主张 × 证据

❖ 读什么 · 怎么看 · 记什么

- ❑ 读什么：正文主干，大体思路，跳过复杂证明与非关键实现细节
- ❑ 怎么看：图表的坐标、单位、误差、样本量、实验条件都要看
- ❑ 记什么：关键观点、疑问、没读的引用，以及主张和证据的对应

❖ 示例 · MapReduce 的四组主张与证据

- ❑ 可扩展到 TB 级 ← 1 TB Sort 基准实验，约 3 分钟完成
- ❑ 具备容错能力 ← 主动 kill 200 个 worker 后任务仍成功
- ❑ 易于使用 ← 用户只需写 Map / Reduce 两个函数

❖ 完成标准

- ❑ 能用三句话讲清楚：主张什么 · 方法核心是什么 · 关键证据是什么
- ❑ 每个主张后面都有可指向的实验 / 图表 / 证明



第三遍 · 验证：虚拟重新实现

❖ 五个动作，从重建走到延伸

- 重建：合上论文，自己把问题、方法、实验回顾
- 挑战：明说的假设有哪些，默认的又有哪些
- 比较：换成我的方案，差别会在哪
- 验证：这些证据真能撑住它的主张吗
- 延伸：记下能改的地方、反例和接着能做的

❖ 实际实现

- 论文的逻辑大多自洽，有时看不出问题
- 动手跑实验才能有第一手经验



用七段式提纲重建论文论证

❖ 七段结构



❖ 以 RDD 论文（NSDI 2012）为例

- ❑ Problem：迭代与交互式分析在 MapReduce 上很慢
- ❑ Prior assumption：跨阶段共享数据必须经稳定存储
- ❑ Insight：工作集留在内存，容错靠血缘重算而非复制
- ❑ Proof：迭代场景快约 20 倍，同时说清明细粒度更新做不了

❖ 评分关注三项

- ❑ Accuracy 主张是否准确还原 · Completeness 是否漏掉关键段 · Clarity 别人能否看懂



什么样的论文值得成为核心文献？

❖ 几个判据

- ❑ 直接相关：直接影响研究问题 / 方法 / 评估 —— 研究迭代系统 → RDD ✓
- ❑ 反复出现：多篇工作共同引用或比较 —— 近年多篇工作都引它 ✓
- ❑ 代表性强：提出关键概念 / 基线 / 范式 —— 提出 lineage 这一新抽象 ✓
- ❑ 可操作：能用于实现 / 复现 / 方案设计 —— 有开源实现 + 公开数据 ✓
- ❑ 时效性：领域里主要团队近三年的相关工作 ✓

❖ 数量原则

- ❑ 典型研究方向的核心文献集合约 3–8 篇
- ❑ 多于 10 篇说明还没收敛；少于 2 篇说明视野太窄

论文总结与思路形成



跨论文比较：做表之前先确定比较目的

❖ 比什么，取决于你想回答什么

- ❑ 找基线：我要跟谁比？任务、数据、指标对不对得上
- ❑ 找瓶颈：它卡在哪？看假设、资源占用和失败场景
- ❑ 选方法：哪个更适合我？比效果、效率、成本和部署难度
- ❑ 找问题：还差什么？看共识、冲突、边界和取舍

❖ 表头设计三条原则

- ❑ 先写出「我要回答的比较问题」→ 再决定列 → 不机械套用固定模板
- ❑ 列的取值应是「事实」而非「主观感受」—— 用 latency / throughput，不用「好」
- ❑ 保持表头稳定，方便不同轮次补充论文，否则每次都要重新对齐列



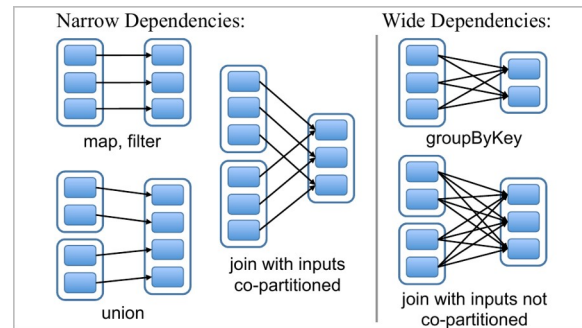
比较表实例 · MapReduce vs RDD

❖ 七个共同维度上的差异

- ❑ 工作负载：大规模批处理 ↔ 迭代 ML 与交互分析
- ❑ 数据共享：跨阶段写入稳定存储 ↔ 工作集可在内存持久化
- ❑ 容错机制：失败任务重新执行 ↔ 沿 lineage 重建丢失分区
- ❑ 主要开销：磁盘 I/O + 序列化 ↔ 内存占用 + 窄依赖约束
- ❑ 抽象层级：Map / Reduce 函数 ↔ RDD 变换 (transformation)

❖ 读表时重点找三样东西

- ❑ 反复出现的假设：所有方法都默认了什么前提？
- ❑ 无法同时满足的目标：效果 vs 效率 vs 开销 vs 适应性
- ❑ 证据覆盖不到的范围：实验只跑了哪些？还有哪些没测？
- ❑ 同一维度上的差异，比逐篇摘要更容易暴露假设和权衡





通过表比较方法的例子

Algorithm	Filtering Type	ANN Index	Filtering Strategy	Attribute Index
SeRF [68]	Range	HNSW	Joint-filtering	Segmented edges
DSG [53]	Range	HNSW	Joint-filtering	Segmented edges
β -WST [22]	Range	VG	Pre-filtering	Segmented subgraphs (Binary search tree)
UNIFY [39]	Range	HNSW	Pre/Post/Joint-filtering	Segmented subgraphs (Cross-subgraph edges) + Skip list
iRangeGraph [63]	Range	RNG	Pre-filtering	Segmented subgraphs (Binary search tree)
FDiskANN-VG [27]	Label	VG	Joint-filtering	Labeled edges
FDiskANN-SVG [27]	Label	VG	Joint-filtering	Labeled edges + Stitched graph
NHQ [60]	Label	NSW/KGraph	None	Joint distance
Milvus [59]	Arbitrary	HNSW/IVF	Pre-filtering	Partition
Faiss-HNSW [20, 42]	Arbitrary	HNSW	Post-filtering	Subset identification
Faiss-IVFPQ [20, 33]	Arbitrary	IVF	Pre-filtering	Subset identification
ACORN [51]	Arbitrary	HNSW	Joint-filtering	Subset identification + Two-hop scan



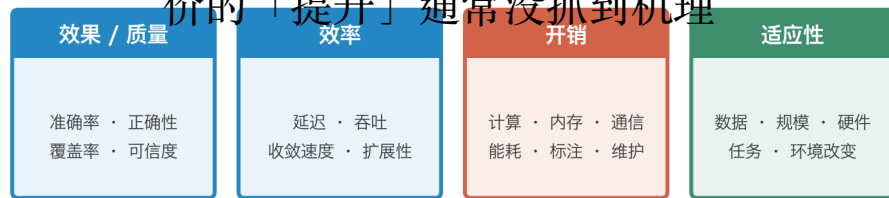
从哪些方面发现问题？

❖ 四个扫描维度

- ❑ 效果 / 质量：做得对不对、好不好 - 准确率、覆盖率、结论可不可信
- ❑ 效率：快不快、能撑多大规模 - 延迟、吞吐、收敛速度、扩展性
- ❑ 开销：要花多少 - 算力、内存、通信、能耗，还有实际金钱成本
- ❑ 适应性：换个环境还灵不灵 - 数据、规模、硬件或任务变了以后还成不成立

❖ 四维权衡示例 · MapReduce → RDD

- ❑ 效率 ↑ (迭代从 30s 到 1.5s) vs 开销 ↑ (占用内存)
- ❑ 适应性 ↑ (支持交互查询) vs 边界收窄 (不适合异步细粒度更新)
- ❑ 任何改进都是权衡 —— 说不出代价的「提升」通常没抓到机理



任何改进都是权衡 · MapReduce → RDD

效率 ↑ (30s → 1.5s) vs 开销 ↑ (占内存) · 适应性 ↑ (交互查询) vs 边界收窄



用 5 要素模板把观察写成可检验的问题

❖ 5 要素：场景 · 已有方法 · 可测不足 · 可能机理 · 研究问题

- ❑ 目标场景：在什么数据、什么规模、什么约束下？
- ❑ 已有方法：现在大家怎么做，靠的是什么假设
- ❑ 可测不足：效果、速度、开销，哪块撑不住？
- ❑ 可能机理：为什么会这样，不只是「是什么」
- ❑ 研究问题：能改善主目标吗，代价又是什么
- ❑ 改进空间：最理想情况下，改进的增益有多大

❖ 示例 · MapReduce → RDD

- ❑ 场景：迭代机器学习 | 方法：MapReduce | 假设：跨阶段经稳定存储共享
- ❑ 方向：在内存中复用工作集 + lineage 容错 | 主目标：效率 ↑，开销可控
- ❑ 不足：重复 I/O 导致高延迟 | 机理：工作集被反复读取但缺可容错的内存复用抽象



回头看和跳出看

❖ 论文不是读完就好，要时常重看之前看过的重要论文

- 看过其他文章，做过实现和实验，对内容有新理解
- 借回头看串联和反思近期研究，形成思路
- “煎熬的周六下午”：只翻论文和笔记，不写代码和论文，强迫思考

❖ 跳出文章看

- 文章的核心并不总是方法，能发表的论文大多能自圆其说
- 跳出方法本身，看它用的数据集（in distribution / OOD）、场景（batch / 流式）和实验配置
- 很多时候机会不在方法本身，而在这些周边内容里

❖ 思路讨论

- 随时和同学导师等讲你的理解和思路
- 表达的过程帮你想清楚



看其他领域的论文和听报告

❖ 了解其他领域，对于做好本领域也非常重要

- ❑ 借鉴：其他领域的工具和方法有可能解决手上的问题
- ❑ 结合：联合其他领域，定义新问题
- ❑ 对计算机领域的重要进展应该保持跟进
- ❑ 公众号、演讲视频都是省力的入门方式
- ❑ 读点方法论的书，比如哲学简史、逻辑学，对建立思考框架有帮助

❖ 听报告和讲座

- ❑ 积极听，不要只是消极参加
- ❑ 积极提问和交流， "There is no stupid question."



准备一场论文调研汇报

- ❖ 覆盖 3–5 篇论文
- ❖ 汇报顺序
 - 问题定义—进展总览—分论文讲解—总结优缺点和展望研究机会
- ❖ 对每篇论文的汇报
 - 先列出发表的会刊、作者和年份
 - 问题导向：围绕论文要解决的几个核心技术挑战讲，其余略过
 - 实验结果要分析透，不能只念数字
- ❖ 总结
 - 要讲清楚可能存在的研究机会
 - 能有初步实验佐证可行性更好